

Performance Analysis of Parallel GMRES Solvers for 2D Fluid Flow on High-Performance Clusters

Sarju Poudel

April 19, 2026

1 Introduction

Large-scale problems in Computational Fluid Dynamics (CFD) and Numerical Analysis often require solving sparse linear systems efficiently. As computational demands increase, serial algorithms become insufficient, making parallel computing essential.

This project focuses on solving a two-dimensional fluid flow problem in a pipe using the Generalized Minimal Residual Method (GMRES) and evaluating performance across different parallelization strategies. Specifically, we compare serial, shared-memory (OpenMP), and distributed-memory (MPI) implementations in terms of runtime, convergence, and scalability.

2 Mathematical Model

We consider steady, incompressible 2D fluid flow governed by the Navier–Stokes equations:

$$(u \cdot \nabla)u = -\nabla p + \nu \nabla^2 u + f \quad (1)$$

$$\nabla \cdot u = 0 \quad (2)$$

where $u = (u_x, u_y)$ is the velocity field, p is pressure, ν is kinematic viscosity, and f is an external force.

For computational tractability, the problem is reduced to a linear elliptic equation (e.g., Poisson-type formulation):

$$-\nabla^2 \phi = g \quad (3)$$

Discretizing using finite differences on a uniform grid yields a sparse linear system:

$$Ax = b \quad (4)$$

where A is a large sparse matrix, x is the discrete solution, and b represents boundary conditions and forcing terms.

3 Numerical Method: GMRES

The system is solved using GMRES, an iterative Krylov subspace method suitable for non-symmetric systems.

At iteration k , GMRES seeks x_k in the Krylov subspace:

$$\mathcal{K}_k(A, r_0) = \text{span}\{r_0, Ar_0, A^2r_0, \dots, A^{k-1}r_0\} \quad (5)$$

where $r_0 = b - Ax_0$ is the initial residual.

The method minimizes the residual norm:

$$\|b - Ax_k\|_2 \quad (6)$$

Convergence is achieved when:

$$\frac{\|r_k\|_2}{\|r_0\|_2} < \epsilon \quad (7)$$

where ϵ is a prescribed tolerance.

4 Parallel Implementations

4.1 Serial Implementation

A baseline single-processor implementation used for correctness verification and performance comparison.

4.2 OpenMP Implementation

Shared-memory parallelism using OpenMP:

- **Fine-grained parallelization:** Parallelizes inner loops (e.g., matrix-vector multiplication), but may incur higher synchronization overhead.
- **Coarse-grained parallelization:** Parallelizes larger computational blocks, reducing synchronization cost.

4.3 MPI Implementation

Distributed-memory parallelism using MPI:

- Domain decomposition of the 2D grid
- Each process handles a subdomain
- Communication required for:
 - Boundary (halo) exchange
 - Global reductions (e.g., dot products in GMRES)

5 Performance Evaluation

The implementations will be evaluated using:

- Wall-clock time
- Number of iterations to convergence
- Speedup:

$$S_p = \frac{T_1}{T_p} \quad (8)$$

- Parallel efficiency:

$$E_p = \frac{S_p}{p} \quad (9)$$

Experiments will vary problem size (grid resolution), number of OpenMP threads, and number of MPI processes.

6 Visualization and Analysis

Results will be analyzed using:

- Convergence plots (residual vs. iteration)
- Performance plots (wall time, speedup, efficiency)
- Error analysis (error vs. grid size)
- 2D contour plots of the computed solution

7 Expected Results

MPI is expected to scale better for large problem sizes due to distributed workload. OpenMP will perform efficiently for moderate sizes with lower communication overhead. Fine-grained OpenMP may suffer from synchronization costs. GMRES convergence behavior is expected to remain consistent across implementations, with minor numerical differences.

8 Conclusion

This project investigates how different parallelization strategies impact the performance of iterative solvers in fluid flow simulations. It provides insight into scalability, efficiency, and trade-offs in high-performance scientific computing.